aws

ROI Training

Anthropic Models on Amazon Bedrock

Module 2:

Claude API Fundamentals and Prompt Engineering

Course Contents

Module 1: Foundations: Generative AI and Claude Models on AWS

▸ **Module 2: Claude API Fundamentals and Prompt Engineering**

Module 3: Data Preparation and Knowledge Bases for RAG

Module 4: Tool Use and Agentic Workflows with Claude

Module 5: Security, Governance, and Guardrails

Module 6: Practical Architecture and Deployment Patterns

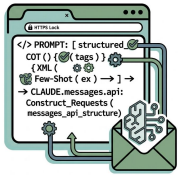
Module 7: Claude Code: Building Bedrock Applications

Module 8: Amazon Kiro and Enterprise Code Generation

Module Objectives

Construct API Requests

Build Claude API requests using the Messages API structure



Apply Prompt Engineering

Use XML tags, chain-of-thought, and few-shot examples effectively

Handle Responses

Implement streaming and non-streaming response patterns

Optimize for Cost

Interpret token usage and design cost-efficient prompts

Amazon Bedrock API Options

Amazon Bedrock

Fully Managed Foundation Model Service

- Amazon Bedrock provides the API surface for invoking Claude on AWS.
- It offers managed access to foundation models from Anthropic and others through a single, unified runtime — no infrastructure to provision, no GPUs to manage, no endpoints to scale.

Choosing Your API: Messages, Converse, or Text Completions

API	Description	Recommendation
Messages API	Claude-native API via InvokeModel	Recommended for Claude
Converse API	Unified Bedrock API across all models	Use for multi-model apps
Text Completions	Legacy completion-style API	Deprecated—avoid

aws

ROI Training

Why Messages API?

We chose Messages API because it offers the most direct control over Claude's parameters, including system prompts, multi-turn conversations, and granular response handling.

Messages API Overview

- Most direct control over Claude's parameters
- Conversational structure with system prompts and messages
- Distinct roles: system, user, assistant
- Multi-turn context maintained in message array
- Bedrock-specific version header required

Messages API Request Structure

- **anthropic_version** required: 'bedrock-2023-05-31'
- **max_tokens** caps response length — required field
- **system** sets Claude's role; **messages** holds turn history

```
{
  "anthropic_version":
"bedrock-2023-05-31",
  "max_tokens": 1024,
  "system": "You are a helpful
assistant...",
  "messages": [
    {"role": "user", "content":
"Hello, Claude."},
    {"role": "assistant", "content":
"Hello! How can I help?"},
    {"role": "user", "content":
"What's the capital of France?"}
  ]
}
```

Multi-Turn Conversation Structure

- Native multi-turn support—no manual prompt concatenation
- **Every** prior message counts toward input tokens
- Stateless apps send a single user message per call

```
{
  "messages": [
    {"role": "user", "content": "Help plan a trip to Tokyo."},
    {"role": "assistant", "content": "I'd be happy to help..."},
    {"role": "user", "content": "What's the best time to visit?"},
    {"role": "assistant", "content": "The best times are..."},
    {"role": "user", "content": "What about cherry blossom season?"}
  ]
}
```

Multi-Turn Context Management

1. Each exchange preserved with role labels
2. Claude maintains context across turns
3. Token costs accumulate with conversation length
4. Summarize older messages for long conversations
5. Set sliding windows or reset at natural breakpoints

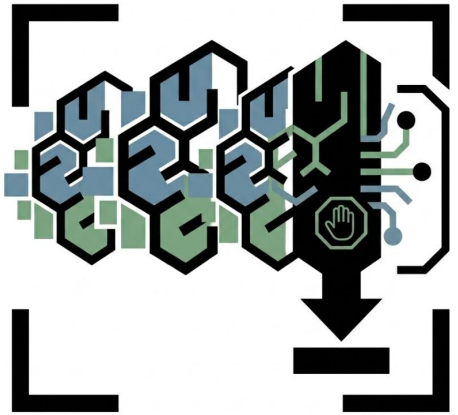
Messages API Response Structure

- **content** is an array of typed blocks (text, tool_use)
- **stop_reason** explains why generation ended
- **usage** reports input/output tokens — track for billing

```
{
  "id":
"msg_01XFDUDYJgAACzvnptvVoYEL",
  "type": "message",
  "role": "assistant",
  "content": [
    {"type": "text", "text": "Paris is
the capital of France."}
  ],
  "stop_reason": "end_turn",
  "usage": {
    "input_tokens": 25,
    "output_tokens": 12
  }
}
```

Understanding Stop Reasons

- **end_turn:** Claude finished its response naturally
- **max_tokens:** Hit your max_tokens limit
- **stop_sequence:** Encountered a custom stop sequence
- **tool_use:** Claude wants to call a tool (Module 5)



Amazon Bedrock Invocation with boto3

- Use service name **'bedrock-runtime'**, not 'bedrock'
- Body is JSON-encoded with the Messages API structure
- Synchronous call — wait for full response, then parse

```
import boto3, json

bedrock = boto3.client(
    service_name='bedrock-runtime',
    region_name='us-east-1'
)

body = json.dumps({
    "anthropic_version":
"bedrock-2023-05-31",
    "max_tokens": 1024,
    "system": "You are a helpful
assistant.",
    "messages": [
        {"role": "user", "content":
"What is Amazon Bedrock?"}
    ]
})
```

aws

ROI Training

Prompt Engineering Techniques

Prompt Engineering Fundamentals

- Prompts are your primary lever for behavior
- No fine-tuning in Bedrock: prompts customize
- Well-structured prompts improve consistency
- Claude responds well to clear, structured input

Technique 1: XML Tags for Structure

```
<context>
Reviewing support tickets for a software company.
Product: enterprise CRM system.
</context>

<ticket>
Subject: Cannot export reports
Body: Export freezes on monthly sales reports...
</ticket>

<instructions>
Analyze and provide:
1. Category (bug, feature request, how-to)
2. Priority (high, medium, low)
3. Suggested response
</instructions>
```

Why XML Tags Work

Why It Works

- Create unambiguous boundaries between content types
- Claude treats each section as a distinct purpose
- Removes confusion: instructions vs. user content
- Cuts ambiguity, improves output consistency

Technique 2: Chain-of-Thought Reasoning

```
<problem>
A company has 150 employees. 40% work in engineering,
30% work in sales, and the rest work in operations.
How many people work in operations?
</problem>

<instructions>
Think through this step by step:
1. First, calculate the engineering count
2. Then, calculate the sales count
3. Finally, determine operations
Show your reasoning before giving the final answer.
</instructions>
```

Chain-of-Thought Benefits

1. Forces Claude to work through problems methodically
2. Reduces errors on multi-step reasoning tasks
3. Makes it easier to verify the logic
4. Particularly valuable for calculations and analysis
5. Trade-off: more output tokens for improved reliability

Technique 3: Few-Shot Examples

```
<examples>
     <example>
     <input>Product broken, support unhelpful.</input>
     <output>{"sentiment": "negative",
      "topics": ["product quality", "support"]}</output>
     </example>

     <example>
     <input>Fast shipping, exactly as expected.</input>
     <output>{"sentiment": "positive",
      "topics": ["shipping", "product match"]}</output>
     </example>
</examples>
```

Few-Shot Best Practices

- Shows Claude exactly what you want
- Establishes format, tone, and scope
- Three to five examples usually sufficient
- Use diverse examples covering edge cases
- Make examples realistic—low-quality examples lead to low-quality outputs

Technique 4: Role Assignment

```
<system>
You are a senior security engineer at a Fortune 500 financial
services company. You have 15 years of experience in cloud
security, particularly AWS. Your communication style is:
- Technical but accessible
- Thorough but concise
- Security-first mindset
- Practical, focused on implementation
</system>
```

Role Assignment Benefits

- Establishes expertise level and domain
- Sets appropriate communication style
- Influences the depth and focus of responses
- Put role definition in the system prompt

Technique 5: Output Format Instructions

```
<instructions>
Respond with a JSON object containing these exact fields:
- "summary": string, 2-3 sentences maximum
- "priority": string, one of ["critical", "high", "medium", "low"]
- "action_items": array of strings
- "estimated_hours": integer

Return only the JSON object, no additional text or explanation.
</instructions>
```

Output Format Best Practices

1. Specify exact field names and types
2. Constrain values to known enumerated options
3. Request ‘only’ the format you want—no preamble
4. Makes parsing responses predictable across calls
5. Essential for production automation pipelines

Demo

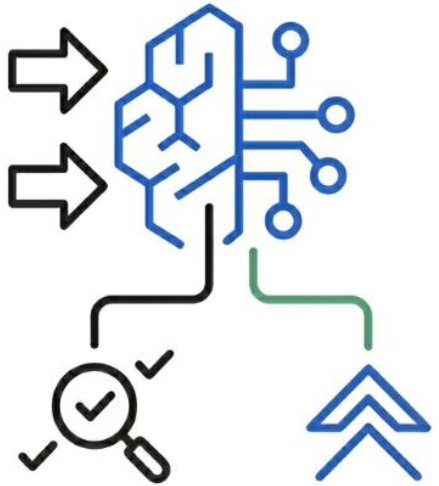
Naive vs. Structured Prompts

- Naive prompt: conversational request, prose response
- Structured prompt: XML tags, JSON output specification
- Compare output consistency and parsability
- Examine token usage differences

Demo: Prompt Comparison Results

Structure pays for itself

- **Naive:** prose, inconsistent, manual parsing
- **Structured:** clean JSON, json.loads() ready
- **Cost:** ~50 extra tokens, huge reliability win



aws

ROI Training

Response Handling

Streaming vs. Non-Streaming

Aspect	Non-Streaming	Streaming
Method	invoke_model	invoke_model_with_response_stream
Response	Complete response after processing	Incremental chunks as generated
Latency	Higher perceived latency	Lower perceived latency
Use Case	Batch processing, APIs	Interactive chat, real-time UIs

Streaming Implementation

```
response = bedrock_runtime.invoke_model_with_response_stream(
    modelId="anthropic.claude-sonnet-4-6",
    body=body,
    contentType="application/json",
    accept="application/json"
)

stream = response.get('body')
for event in stream:
    chunk = event.get('chunk')
    if chunk:
        data = json.loads(chunk.get('bytes').decode())
        if data['type'] == 'content_block_delta':
            print(data['delta']['text'], end='', flush=True)
```

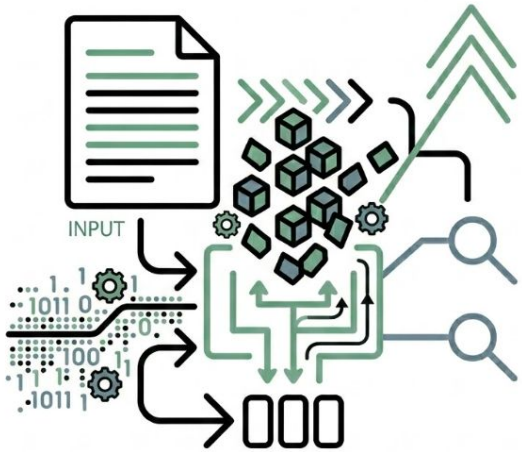
Streaming Event Types

1. **message_start:** beginning of the response
2. **content_block_start:** start of a content block
3. **content_block_delta:** text chunks — display these
4. **content_block_stop:** end of a content block
5. **message_stop:** response complete

Token Usage Fundamentals

Tokens drive every cost decision

- 1 token is roughly 4 characters or 0.75 English words
- Input tokens: prompt + system + conversation history
- Output tokens: Claude's generated response
- Both cost — output tokens are 5x more expensive



Token Pricing Reference

Model	Input per MTok	Output per MTok
Opus 4.6	$5.00	$25.00
Sonnet 4.6	$3.00	$15.00
Haiku 4.5	$1.00	$5.00

Prompt Optimization Strategies

1. **Right-size max_tokens:** Don't set 4000 when you need 200
2. **Be concise in system prompts:** Every token costs money
3. **Use structured output:** 'JSON only' prevents verbose text
4. **Consider model selection:** Use Haiku for simple tasks
5. **Manage conversation history:** Summarize or truncate old msgs

Cost Comparison Example

Approach	Model	Input Tokens	Output Tokens	Monthly Cost
Verbose prompts	Sonnet	500	200	$990
Optimized prompts	Sonnet	200	50	$285
Optimized + Haiku	Haiku	200	50	$75

Error Handling Best Practices

```
import botocore.exceptions

try:
    response = bedrock_runtime.invoke_model(...)
except botocore.exceptions.ClientError as e:
    code = e.response['Error']['Code']
    if code == 'ThrottlingException':
        time.sleep(backoff_delay)
        retry()
    elif code == 'ModelTimeoutException':
        log_error_and_alert()
    elif code == 'ValidationException':
        log_error(e.response['Error']['Message'])
```

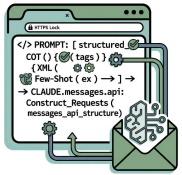
Common Errors and Solutions

- **ThrottlingException:** Exceeded quota — implement exponential backoff
- **ValidationException:** Malformed request — check error message
- **ModelTimeoutException:** Request too long — reduce prompt or max_tokens
- **AccessDeniedException:** IAM issue — verify policies
- Always implement retries with exponential backoff for transient errors

What We Covered

Construct API Requests

Build Claude API requests using the Messages API structure



Apply Prompt Engineering

Use XML tags, chain-of-thought, and few-shot examples effectively

Handle Responses

Implement streaming and non-streaming response patterns

Optimize for Cost

Interpret token usage and design cost-efficient prompts

Module 3 Is Next!

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

▸ **Module 3: Data Preparation and Knowledge Bases for RAG**

Module 4: Tool Use and Agentic Workflows with Claude

Module 5: Security, Governance, and Guardrails

Module 6: Practical Architecture and Deployment Patterns

Module 7: Claude Code: Building Bedrock Applications

Module 8: Amazon Kiro and Enterprise Code Generation